两万字长文｜手把手带你趟过 AI Coding 深水区：编码让位，人退到哪里

原创 腾讯程序员 腾讯技术工程 Aug 27, 2026, 2:38 AM 北京 466 listened



作者：黄迅， PUBGMobile后台

> 百万行 Lua，自研框架，模型几乎没见过——代码让给 AI 之后，人退到判断上，把取事实的活交给机器。

前言：AI 已经能端到端交付了

这个季度我们做了一个内部平台，给我们的 coding agent 做支撑：七个服务，三个月，一千八百多次提交，十六万行生产代码、十一万行测试代码、八万多行前端，**没有一行是人写的**，设计文档也全部由 AI 落笔。

这个平台是对照组：通用技术栈，全新项目，没有历史包袱，对 AI 最友好的那种环境。我们先在对照组里把"代码全让给 AI"真正跑了一遍。这也意味着，我们的业务代码离 AI 端到端交付，已经不远了。

人做了什么？做了大量的决策。

（一）人的工作量没有减少，只是从落笔挪到了拍板

我原本以为 AI 全权接过代码之后，人会轻松下来。实际上完全没有。每天要读的东西一点没少，只是从"读代码"变成"读方案、读结论、读证据"；要拍的板一点没少，只是从"这段怎么写"变成"这条路走不走、这个失败算不算问题"。

仓库里现在留着七十多篇设计文档，全部由 AI 落笔，每一篇的取舍都是人定的。但真实产出远不止这个数——**海量的 spec 文档最后被我删掉了，删完项目照跑，没有任何损失。** 留下来的这些，还有一部分是我专门让 AI 照着已经落地的实现反推重写的：文档跟着代码走，不是代码跟着文档走。这里有个区分值得说清——**动手前那批用来对齐的 spec，用完就可以扔；事后沉淀的设计说明得留着，因为它是给下一个要改这块代码的人建立心智模型用的。** 两者长得像，寿命完全不同。

这件事顺手回答了我在上一篇里没想透的一个问题。SDD 流行之后，很多人把 spec 文档当成核心资产。可如果文档真是核心，那我删掉的就该是核心，项目该出事——它没有；而核心工作既然已经被 AI 接走，人的工作量该降下来——它也没降。

两头都对不上，说明核心从来不是 spec 这个载体，而是产出它的过程中被迫想清楚的那些事。**spec 的价值在于逼你在动手前把问题定义清楚；一旦想清楚了，文档本身是可丢弃的。**真正不可替代、也无法委托给 AI 的那一步，是想清楚它——brainstorming。

（二）AI 犯错不是概率问题，是规模问题

即便在那个最友好的环境里——Go 加 TypeScript，全新项目，工具生态齐全——三个月下来，它照样攒了一份事故清单。资产入库卡在中间状态四次，每次都在提交一个空 commit；覆盖率看板把"采集失败"和"没有数据"显示成同一个东西；重试队列里的僵尸任务把整条调度锁死；需求提取上线第一晚积压四千多条。

这些事故让我把一个模糊的直觉想清楚了：**不管什么技术栈，AI 产出都需要大量保障工作，因为出错的绝对数量不由单次准确率决定，由累积量决定。**迭代次数足够多、产出代码足够长，再低的错误率也会积成一堆真实事故。而人的审查能力，一点都没跟着涨。

所以那种说法——模型越强，harness 就该越薄——是想当然。模型变强只会让产出更多、更快，人这一端的上限一动不动，缺口只会更大。

（三）有一类错，AI 特别容易犯

这些事故里有一类特别值得单独说：不是功能不通，而是失败不可见。

AI 写得出完整的状态机，但最初漏掉了"合并失败"这一态，于是失败没有名字，只能卡在"正在合并"里原地打转。AI 写得出覆盖率看板，却把采集失败和真的没覆盖显示成同一个 0%。AI 写得出重试队列，却让僵尸任务把调度锁死。

AI 交付功能的能力，明显强于它交付"让失败可见"这种纪律的能力。因为前者是需求，后者是价值观——而正如后面会讲到的，AI 的训练是应试式的，它目前还理解不了价值本身。

这就是人的决策不可替代的地方。人在这些事故里做的判断是"这个失败必须有名字""这两种 0% 必须分开"。它们不写在任何需求里。

（四）三个发现指向同一句话

> **机器提供事实，人做判断。**

它是一把可以直接用的尺子：手上这件事，是事实，还是判断？是事实就交给机器，别让人去核对，也别听 AI 声明；是判断就留给人，别指望机制替他拍板。

这三个发现都是对照组给的。我们真正要天天干活的地方是另一边：百万行 Lua，自研框架，市面上的模型几乎没见过，历史债一层叠一层。连对照组都要这么兜底，这边只会更需要。

所以下面五件事都是在这边摸出来的，它们是一条递进的链：AI 那一端得顶得住，提示词顶不住的沉进框架；框架顶不住的，继续往下沉到 runtime；干得好不好，要能量出来；机器顶住了、量得出了，才知道人该退到哪里；最后把这些判据变成每天都在跑的流程。每一步的动作其实是同一个——把某个环节从"信声明"搬到"信事实"。

全文大纲

部分	我们曾经信	后来改信
一、提示词的尽头是基础设施	再强调一遍，AI 就该记住了	注意点堆在提示词里模型会变笨，沉进框架才越厚越强
二、编排的尽头是 runtime	官方机制够用，skill 能补上	机制残缺补不动，runtime 必须可审计、可干预
三、分数不可信，问题在题集	评测得建成一套完整的系统	判据必须可执行，工程量不代表可信度
四、人退到决策点	人逐行 review 兜得住	有些事实只有机器取得到，门禁 AI 绕不过
五、把判据变成流程	一次会话做到位，信 AI 的自跑报告	大环套小环多重检查，环与环之间只传事实

以上五条是本文的核心内容，也是五个故事，下文会按这个顺序逐条讲。

第一部分：提示词的尽头是基础设施

> **本部分要点**
>
> - 提示词的边际收益会耗尽：注意点越堆越多，模型注意力越碎，主任务反而越差。
> - 约束要沉进框架：声明式 mock 自动还原、trace 超时分析包，是用框架“取消这个错误的存在”，把 AI 的注意力还给主任务。
> - 报错通道本身就是提示词注入通道——最好的提示词是恰到好处的反馈；但带错方向的反馈比没有更糟。
> - 下沉的标准：判据是确定事实才做硬门禁，是代理指标最多做提醒；信号要从外部取；误报会逼人把代码写得更差。

先看 AI 那一端怎么顶得住。这一部分讲一个所有做垂类 agent 的团队都会撞上的现象：**提示词的边际收益会耗尽。**

AI 时代，几句提示词就能做出像模像样的 demo。但往生产可用做深，最后都会走到同一个地方：基础设施。

以我们的测试 agent 为例。先交代背景：项目技术栈特殊，我们专门开发了端到端测试框架，给 AI 用之前已经内部用了数月，框架成熟之后才交给 AI。测试 agent 的提示词和流程编排又打磨了数月——然后很快进入瓶颈期：不管怎么调提示词，都调不出初期那种立竿见影的效果。

我当时的第一反应还是继续写提示词。 措辞再强硬一点，规则再细一点，例子再多给一个。这个方向撞了很久的墙，直到我把手头所有测试会话摊开来分析（用的是自研会话分析工具 tracesift），才看清剩下的问题在提示词和流程层面根本解不掉。举两个例子。

（一）例一：mock 还原

测试的经典范式是 setup/teardown：开始前 mock，结束后还原。提示词里反复强调"setup/teardown 必须成对出现"，AI 仍有约一半概率漏掉 teardown。后果是用例互相污染，整个验证过程乱到不可控。

常规思路是加 lint hook，但这个问题很难用规则式 lint 查出来，AI 也仍有概率绕过。后来我换了个思路：为什么不在框架层解决？就像自动 GC 一样，我给框架补了**声明式 mock 和自动还原**——mock 声明出来，用例结束框架自动还原。之后 AI 在这里几乎不再犯错。

写测试要同时顾的点太多，这个任务对 AI 其实很重。**用框架消掉一整类错误，等于把 AI 的注意力还给主任务。**

（二）例二：trace 链路分析

AI 跑测试最常撞的问题是链路超时。这类问题要读大量代码、追调用链，在写跑测这种本就繁重的任务里再塞进重度分析，AI 经常耗时极长，还有一定概率上下文过载、注意力涣散，彻底解不出来。

我重构了框架内的 trace 实现：**链路超时报错时，自动返回准确原因和链路分析包。**

裸超时只能告诉你"没等到回包"。所以框架在超时那一刻反向探测各个服务的队列和限频登记表，按探测到的现场把超时分类：还在排队、出队了但业务代码没回包、在分发层被限频丢掉、回包到了但当时没人接。每一类后面跟的不是现象描述，是下一步动作——比如"检查前置请求是否用完了限频配额"，连该去看哪个配置字段都点出来。分析包里还有一节叫代码执行路径：框架打开业务源码，把这条链路最后执行到的那几行原样贴进报错，末行标一个箭头。AI 不必再猜卡在哪，它直接看到卡在哪一行。

这个设计有个值得单独说的副产品：**报错通道本身就是提示词注入通道。** 顺着它可以返回任何 AI 需要的信息，而且时机精准——它正好在 AI 卡住的那一刻到达。

最好的提示词，是恰到好处的反馈。

但这条通道也会反噬。框架里曾经有一个告警：AI 等一个不在协议表里的协议名，就提醒它一句。后来我们把它删了，理由写在文档里——协议表是按线上流量生成的，冷门协议本来就收不全，登陆窗口期的协议更是全都不在表里；于是这个告警信噪比很差，还会把 AI 带偏。注入的信息不是越多越好，会带错方向的反馈，比没有反馈更糟。

（三）两个例子的共同点

解法都不在 agent 层，在 agent 底下的框架层。提示词层面你只能"叮嘱"，框架层面你可以"取消这个错误的存在"。

最直接的证据是一条纪律的消失。自动还原上线之前，我们靠人肉纪律维持秩序：会改脏环境的那批用例必须排到最后跑，跑完还得重启服务。等自动还原和静态闸门都到位，交接文档里写下的是——这条纪律"自此作废"。不是 AI 变听话了，是这类错误不再可能发生，纪律没有了存在的理由。

（四）注意点加不下去了

测试 agent 的那个 skill 叫 ut-tester。SKILL.md 四百多行，排坑、模板、注意点铺了十几份文档。2 月到 6 月每个月都在往里面加；7 月之后几乎停了。不是做完了，是加不下去了。注意点越多，模型要同时记住的事越多，注意力越碎，写测试这个主任务反而越差。

这就是"harness 越厚，agent 越笨"真正成立的地方。开发环节充斥的就是这种约束：skill、SOP、流程说明，靠模型按你写的走。它要有——领域里那些稳定的事实、能力边界，不写模型不知道。但它有上限。再厚，厚的是模型的工作记忆。

打补丁这一档尤其不要写。模型走错一次，就加一条"以后必须走这条路"。我们写过"严禁 mock ss，只有一种正确写法"，八天后对着仓库里 529 个存量单测才发现，这条从落笔那天就和事实相反。它不是用久了才坏，是生来就在拿有限规则覆盖无限问题。

同一时期力气转到了 ut 框架。6 月框架侧的提交大约是 skill 的四倍，7、8 月还在加覆盖率制品、把 trace hook 往更底层沉、给隔离运行补协议。ut-tester 最后一次实质改动，标题就是"对齐 UT 框架近期改动"。mock 自动还原、超时分析包，都是把原来写在 skill 里的叮嘱从模型脑子里挪走。挪走一件，主任务就清静一点。

所以有两种"厚"，别混。写在提示词里，越厚模型越笨；写进框架，越厚模型越省心，主任务上反而更强。验证是第三件事，发生在产出之后：核它交出来的测试是不是真能抓 bug、报告有没有撒谎。那是后面评测和验收的事，不是开发环节的 SOP。

生产级 agent 建设，最终都归到基础设施建设。AI 时代基础知识不会过时；相反，框架设计、编译原理、协议栈这些硬功夫，正是决定你的 agent 能不能走出深水区的东西。

（五）那是不是什么都该往下沉？

约束型的注意点不该无上限地堆，但不等于能沉的都该立刻沉。判断标准就摆在我们自己的一个 hook 文件里——同一个拦截器里的两条规则，待遇完全不同。

一条是"禁止读 protocols 目录"，判据是路径前缀比对：目标路径在不在那个目录下，要么在要么不在。它至今是硬拦截，AI 撞上就是撞上。

另一条是"编辑 `.lua` 之前必须先读编码规范"。它一开始也是硬拦截，判据是扫会话记录，看 AI 有没有读过那两份规范文件——上线两周后降成了警告。**因为这个判据不是事实，是代理指标：** 读过不等于遵守，没扫到也不等于没读过。它自己还翻过一次车，代码注释里留着结论——主会话累积了所有历史 Read，会把"已读"这个状态污染给之后的每一个子 agent。

判据是确定的事实，才能做硬门禁；判据是代理指标，最多做提醒。 这跟前言那把尺子是同一把，只是换了个用法：那里分的是交给机器还是留给人，这里分的是做成机制还是留在提示词。

顺着这条标准，还有两件事值得说。

一是信号要能从外部取到，不能靠 AI 自述。 前言那个平台的测试跑批上有一道硬失败：脚本报告成功、报告文件也生成了，但覆盖率是 0/0 行——直接判失败。它防的是一类很隐蔽的假测试：AI 把被测函数的内部逻辑抄出来，用本地变量重算一遍再断言。断言全过，覆盖率零，测不出任何回归。这种错误提示词防不住，因为 AI 自查也会认为自己做对了；只有从外面看"被测函数的行一次都没被执行到"，才戳得穿。

二是门禁误报的代价比想象的高。 ut-lint（我们写的 Lua 测试静态检查）里有两条规则明知易误报，我们始终没敢改成阻断，只在 reviewer 的提示词里要求：每条 lint 结果都要读源码二次确认，采纳还是判为误报，都得写明理由。因为误报的真实代价我们见过——改进其中一条规则时，目的写得很直白：不再为了过 lint 把 helper 拆开内联。门禁误报的后果不是浪费一次提醒，是逼着人和 AI 为了过检查，把代码写得更差。

最后补一件不太顺利的事。ut-lint 最早只写在 reviewer 的提示词里，要求它评审前先跑一遍——还在约束层，靠模型记得去跑。想让它往下沉，第一个想法是做成 SubagentStop hook：子 agent 一结束就自动跑，结果直接塞回上下文。做不成——Claude Code 和 codebuddy 的 hook 协议里，SubagentStop 只支持"阻断并给出理由"，不支持往上下文里注入内容。我们要的是提示，协议只给阻断。于是方向改成把 lint 触发塞进测试执行脚本本身：跑测试就自动跑 lint，脱离 agent 框架，CI 也一起受益。这条还没落地，但它比 hook 那条路更彻底——同一套规则，从"叮嘱模型去跑"变成"环境里跑测试就会跑"。

这件事透出一个更麻烦的问题：**能不能往下沉，一半不由我说了算，取决于我脚下那层给不给接口。**

第二部分：编排的尽头是 runtime

> **本部分要点**
>
> - workflow-engine 换来了稳定，但 token 消耗、体验、主 agent 闲置三个问题越来越扎眼。
> - 两项 agent teams 实验：完整流程太脆没推广；收窄到测试流程的 /module-test 跑稳了，上百次失败会话熬成 agent-teams skill。
> - skill 能缓解机制的不完善，缓解不了机制的残缺；三起黑盒事故证明 agent loop 的每个环节都必须可审计、可干预。
> - 编排问题追到根上是 runtime 主权问题；选型的真问题不是“选哪个产品”，是主权的边界划在哪。

框架层能解的都解了，剩下的要么解不掉，要么像 ut-lint 那样被底座的口子卡着。编排怎么走、上下文怎么压缩、工具参数怎么拼接——这些到底在哪一层？

（一）workflow-engine 的三个遗留问题

上一篇收尾时，我们的多 agent 编排停在 workflow-engine：主 agent 做成状态机，按固定流程 spawn 子 agent。当时的结论是"真正贵的不是 token，真正贵的是失控"，所以坚定选了稳定。

跑了一个季度，稳定是换来了，但三个问题越来越扎眼：

1. **token 消耗大**
   子 agent 每次从干净上下文出发，同一份代码反复探索。多轮循环里尤其明显——review 打回三轮，implementer 就把相关代码读三遍。
2. **体验不佳**
   编排逻辑复杂，上手门槛高；子 agent 干活的过程不透明，用户没有掌控感。
3. **主 agent 浪费了**
   它只维持秩序，最值钱的能力——直接和用户对话——闲置着。

所以这一季度我们重新打开了当初关掉的那扇门。不是因为"失控比 token 贵"这个判断错了，而是因为光有"稳"已经不够用了。两个方案并行推进：

- **方案一：**需求分析、架构设计这两个交互最重的环节收回来，主 agent 亲自做；拉子 agent 时优先续接（resume）上一轮的，不重新 spawn。好处有三：交互最多的环节，体验向单 agent 看齐；主 agent 派活时能带上更多有效上下文；子 agent 带着上轮记忆接着干，不用重新探索。这条路风险低，已经落进现在的 /dev 流程。
- **方案二**：Claude Code 独有的 agent teams。本质是网状的多 agent 通信：成员常驻，互相直接发消息，共享任务看板。常驻天生适合多轮循环，网状通信把编排的灵活度拉满。单看理论，这是解上面三个问题的绝佳方案。

方案二风险大——直到今天，agent teams 除了官方文档几乎没有公开实践，官方也一直把它标为实验特性。这条路由我来蹚。

（二）第一项实验：把整个开发流程搬上 Team

五月底我做了第一项实验（`experiments/2026-05-29-team-driven-dev`），思路很激进：

- **去掉 `/dev` 命令和 workflow-engine 状态机**
  换成两个 skill 串联：brainstorming（主 agent 交互式设计，产出 arch.md 和 plan.md）→ team-execution（Team 驱动执行）。
- **产物驱动状态**
  不用 state.json：arch.md 不存在就从头开始，plan.md 存在就直接执行。中断恢复靠查磁盘产物，不靠状态机。
- **七个角色进 Team**
  planner、implementer、reviewer（可多实例）、test-planner、test-engineer、explorer、guardian。审查循环点对点直连——implementer 和 code-reviewer 互发消息对审，不经主 agent 中转。
- **guardian 异步守护**

一个只盯"用户说的话和主 agent 的理解是否一致"的旁观者，防长对话里曲解需求。

设计在纸面上很美，实跑下来，提交历史里全是补丁。踩的坑分四类，从浅到深说。

第一类：模型根本不认识这套机制。 Anthropic 没把 agent teams 训练进模型，所有行为全靠提示词现教。于是：主 agent 拿到需求直接开聊，忘了先建 Team——只好在 skill 里加"STEP 0 强制前置，不可跳过"；工作流目录命名在 kebab-case、CamelCase、中文之间摇摆——只好把占位符直接写死成 `snake_case`；guardian 看到"通过 tracesift 工具读取日志"，跑去内置工具列表里找 tracesift，找不到就放弃走兜底——只好改成"tracesift 是 shell 命令，用 Bash 调用"。每一条都是一次失败会话换来的提示词补丁。

第二类：中断活锁，栽得最狠的一个。 一次完整实跑（约 2.5 小时的真实需求）后复盘，planner 反复"莫名 idle"，产不出 plan.md。用 tracesift 解析会话记录，才挖到根因：主 agent 大约一分钟催一次 planner，而 planner 单轮热身——重载方法论 skill、读 arch.md、补做被打断的验证——就要超过一分钟。每条催促都打断它在途的工具调用，本轮作废，输出一轮比一轮少：580 token → 278 → 最后只剩 1。planner 在会话记录里留下原话："我的 skill 加载和验证查询又被消息打断了。"更糟的是，主 agent 把这误判成"Team 在 resume 后失效"，于是重建团队接着催，活锁就这么自己养活自己。

这次复盘换来三条铁律，后来都进了 skill：**不催正在工作的 agent，只认显式 DONE/BLOCKED 信号，idle 是正常态**（我们叫它 PROTOCOL-A）；planner 这类角色改成任务来了一次性 spawn、自包含干完；测试断言从架构文档的验收标准提前冻结，实现完成后只填占位符，防止有偏差的实现反过来污染测试。

第三类：成员的生命周期靠不住。 进程内的成员随主进程消亡，会话续接后活体状态全丢，主 agent 却还在给不存在的成员发消息。对策只有一个：一切以磁盘产物为准，失联就拿磁盘上的 arch.md / test-plan.md 无损重建成员，不指望常驻永远活着。

第四类：机制本身反直觉。 你其实强杀不了一个不配合的成员——强杀工具要 taskId，你 spawn 拿到的却是 agentId，两者不互通。成员不配合关停时，解散团队的调用会一直失败，唯一解法是手动改 `~/.claude/teams/` 下的 config.json，把成员摘掉。任务一旦指派了归属成员，成员之间的任务窃取就关掉了，它卡死之后名下的活没人能接。这些没有任何文档写，全靠翻源码和实机踩坑。

完整流程的编排最终没有推广——不是跑不通，是太脆，我没信心交给其他同学用。但它留下了一样值钱的东西：**agent-teams skill**——从上百次失败会话里熬出来的运维知识库。内容就是把上面那些坑一条条理清楚：怎么读懂成员状态（idleReason 三态）；失败怎么按 failureReason 语义分流——限流就退避重试同一成员、绝不换人，认证失效就停手找人；卡死判据用"超时没消息"，不看状态符号；防消息风暴和中断活锁的通信纪律。

注意这几条纪律的共同点：**全部是"不信声明，信事实"。** 不信状态符号，信超时有没有消息；不信活体成员，信磁盘上的产物。

（三）第二项实验：收窄战场，只在测试流程用 Team

完整流程靠不住，那就收窄：只把测试流程搬上 Team，并且**严格限定通信方式和流程**。这就是现在的 `/module-test`：

- 主 agent 亲自写 spec——spec 是"自带答案的交接物"，协议锚点、读回方式、验证点矩阵全部代码核实过，不许留空。
- Team 里只有两个成员：test-engineer（写计划、写代码、跑测试）和 reviewer（独立审查）。两人点对点直连互审，主 agent 不中转、不代审，只按 agent-teams skill 监督进度、容错兜底、三轮不收敛就升级给用户。
- spec 的验证点矩阵是审查的权威基准，早早冻结。测试红了且对应 spec 验证点，按代码 bug 处理，绝不软化断言求绿。

这套编排我跑了几十遍各类任务。有 agent-teams skill 和 test-generator skill 加持，整个流程能比较稳定地走完，还保住了 agent teams 最大的优点：**我随时能和任意 agent 直接对话，自由干预任何环节**——这正是 workflow-engine 给不了的掌控感。

但它有个编排层面无解的缺点：reviewer 常驻，复审时免不了被自己上一轮的意见带着走，只盯上轮问题，审不全。彻底解决只有一条路：每轮评审都 spawn 新 agent。权衡评审时长和 token 消耗，我们最终留下了常驻 reviewer——明知有损的取舍。

两项实验下来，我的判断是：agent teams 方向是对的（网状通信、常驻成员、自由干预），当前实现是残缺的——模型没被训练过这套机制，工具的提示词和行为对 LLM 不友好，建在它上面的编排逻辑，怎么写都脆。**skill 能缓解机制的不完善，缓解不了机制的残缺。**

那"残缺"到底缺在哪？缺在我们对 runtime 没有主权。空口无凭，接下来三件真实事故就是证明。

（四）三起黑盒事故

第一件：Claude Code 的遥测事件。 claude code 会用隐蔽手段检测使用者是否来自中国，命中后在提示词和工具调用里投毒。细节这里不展开，但它直接动摇了"把生产流程建在一个黑盒 CLI 上"的信任基础。

第二件：CLI 写长中文文件时的 UTF-8 损坏。 我们用 CLI 批量生成中文技术文档，37 份产出里 36 份损坏、累计 370 处 U+FFFD，根因在 CLI 拼接工具调用参数时按字节切断了多字节字符（第四部分讲验收时还会从另一个角度撞上它）。这段代码在黑盒里，我改不了。

第三件：压缩 bug 导致的 11 小时空转（完整复盘见团队博客《一个 agent 空转 11 小时的复盘》）。一个只读探索任务跑了 11 个小时、6556 次工具调用、1.23 亿 input token，一无所获。四个因素叠在一起：max_turns 限额没生效；压缩阈值卡在 100K，模型 200K 的窗口只用了一半；压缩摘要把"生成摘要"这条指令当成了用户请求，对恢复最关键的"当前进度""待办事项"两节经常只剩标题；于是每轮压缩后 agent 都失忆，从同一组关键词重新 Grep。203 轮压缩，就是 203 次从头再来。

（五）合流：自建 agent runtime

两条线到这里合上了。编排实验证明：机制残缺，skill 补不动，要改只能改 runtime 本身。三起事故证明：**agent loop 里的每个环节——上下文怎么压缩、工具参数怎么拼接、限额怎么执行、遥测发了什么——都必须可审计、可干预。** claude code 和 codebuddy 都不开源，开发者只能靠提示词和 hook 从外面伸手，出了问题，定位都要靠逆向会话记录。对垂类场景，这种定制能力太弱了。

自建 agent runtime，有必要。

那成本呢？会不会重复造轮子？把选项摆开：

- **从 trpc-agent-go 这类框架写起**：真的重复造轮子，agent loop、工具协议、上下文管理全得自己做。
- **基于 trpc-agent-go 做薄封装**：必然有别的团队在做同样的事——说实话我的第一反应也是它，笑。
- **基于 claude/codebuddy 的 agent SDK**：底层还是黑盒，三起事故一件都躲不掉。
- **codex CLI / SDK**：开源，可定制，但体量大，理解成本高——而且它的开源没有看上去那么彻底，下面单独说。
- pi agent：MIT 开源，自称"最小 agent harness"，系统提示词不到 1000 token，子 agent、plan mode 这类功能故意不内置——要就自己用 TypeScript 扩展写，扩展能碰到工具、命令、事件和整个 TUI，还能打成包分发。它挂在首页的卖点里有一条正对我们的痛处：极简系统提示词加可扩展性，让你真正做上下文工程，控制窗口里放什么、怎么管理。边界也摆得明白：能改的是它预留的那些扩展点，agent loop 本身在 core 里，是代码，不是插件。
- **DeepSeek Harness**：写这篇的时候才开源的 v0.1 开发者预览，MIT 协议，"一切皆插件"——模型、工具、技能、会话、沙箱、存储、循环、调度、UI 全部是插件，底下的 Cordis 元框架只负责插件的加载卸载和依赖关系。不动它的源码，就能替换其中任一能力，包括 agent loop 自己。

codex 值得多说一句，它暴露了一个更隐蔽的问题：**开源不等于主权**。它新版的多 agent 通信把子 agent 的任务载荷加密，密文只有 OpenAI 服务端解得开——父子 agent 都接 OpenAI 才能通，子 agent 换别家模型，密文就跨不过去；本地调试连真实任务文本都看不到；第三方想提供兼容的 Responses API 服务，也过不了这道解密。名义上支持多 provider，核心协议却焊死在一家服务上。说难听点，这是假开源：仓库开着，协议锁着，主权还是别人的。

codex 这条顺带把选型的真问题挑了出来：不是"选哪个产品"，而是**主权的边界划在哪里**——落到实处，就是插件化能划到哪一层。三起事故就是量这条边界的尺子，它们要求的可替换性一条比一条深：

- **工具参数怎么拼接**
  ——把 UTF-8 按字节切断这种事，我得能自己接管那段代码。这是最外层，多数底座都够得着。
- **上下文被注入了什么、遥测发了什么**
  ——模型看到的每一样东西，都得能按来源回看。这是审计的底线。

- **上下文怎么压缩**
  ——压缩策略得能整个换掉，而不是去猜它的阈值卡在多少、摘要模板长什么样。这条最深，因为它长在 agent loop 里面。

对照那起 11 小时空转：根因是压缩摘要把"生成摘要"当成了用户请求、对恢复最关键的两节只剩标题——这是靠逆向会话记录才拼出来的。如果每一次注入本来就有据可查，这种复盘不必从考古开始。

这把尺子一量，pi agent 和 DeepSeek Harness 的差别就不是新旧，是边界划在哪：pi 把 loop 和压缩策略留在 core 里，DSH 连这两样都推成插件，元框架只管装卸。审计那条 DSH 也接得住——模型看到的一切都进 append-only 会话日志：系统提示词、思维链、工具调用与结果、子 agent 调度，以及**每一次上下文注入**，都能按来源回看；恢复、分叉、回放共用同一份事件流。

架构上 DSH 更彻底，但选型不能光看架构。这两个选项在我手上的证据强度差得远：pi 我跟了几个月，也上手试过，虽然试出来的东西还微不足道；DSH 我手上只有一篇官方博客——"一切皆插件"目前还只是它自己的说法，我还没亲手量过。

这里正好用得上本文那把尺子：**声明归声明，事实归事实。** 顺序也就不是偏好问题了：

- **pi agent 先动**：几个月的接触打底，扩展门槛低，而它自我定位的重心正是上下文治理，恰好压在我们的伤口上。垂类扩充从改提示词、加工具开始，一路撞到 core 的边界就记一笔账——记下来的每一笔，都是下一步的需求清单。
- **DeepSeek Harness 跟进**：按"一切皆插件"的设计思路，它是可预期更好的底座，连 loop 都能换掉。但"可预期"这三个字得自己去验，不能听它说；核心插件和基础接口还在快速迭代，现在就把整条生产流程押上去，等于拿生产环境替它做测试。

要守的原则很清楚：**元框架只管装卸，能力全部做成可替换的插件**。路径也清楚：从最小 harness 起步，撞到墙就换掉撞墙的那块，不必一开始就背上整个框架的复杂度。真正的垂类扩充还没排上工期——那是工期问题，不是要不要做的问题。怎么走，留给下一篇。

编排问题追到根上，是 runtime 主权问题。这个认知不是拍脑袋想的，是两项实验、三起事故加一次源码审计，一起买来的。

第三部分：分数不可信，问题在题集

> **本部分要点**
>
> - 十七万行的评测平台只留下两个分数；16 次跑数的小实验改掉了一个至今在用的策略——差别不在工程量，在判据能不能执行。
> - 业界调研的结论不留情面：有论文支撑的方法全站得住，唯一自研的挖空回填恰恰塌掉。
> - V2 把主指标落在变异击杀率：不问“找没找到我藏的东西”，改问“你写的测试实际能发现多少 bug”。

> - 评测的核心工作是构建题集，不是搭平台；凡是能脚本化的环节，都不值得当成工程去建设。
> - 评测能帮你改进 agent，但成本业务团队很难扛：先想清楚目的，再决定上不上。

前两部分，AI 那一端顶住了：测试 agent 能干活，编排追到了 runtime。跟着的问题就是：它写出来的测试到底有没有查错能力，能不能量出来？这一部分回答它。

评测，谁都说该做，很少有人讲怎么做，更少有人算过账。这里摆两次推导：一次撞了墙，一次走通了。两次的投入差了两三个数量级，可教训落在同一个地方。

（一）先说撞墙的那次

先说清一件事：做这件事的判断本身是对的。那时候 agent 的能力确实在涨，但涨了多少、哪里还弱，全凭感觉——没有度量就没法迭代，这是当时团队里的共识，我也是其中一票。方向没错，错的是形态。

今年三月底到七月中，我们给编码 agent 建过一套很重的评测系统。107 天，评测目录累计增加十七万行（其中三万多行是题集里的参考实现）。建成的东西相当完整：从建隔离工作区到上传 trace 的六阶段流水线，四层加权评分，React 加 FastAPI 的 Web 平台（连登录鉴权都做了），为了看 trace 自部署了六个容器（Langfuse 加它依赖的 PostgreSQL、ClickHouse、MinIO、Redis），跑完自动推企微，综合分低于 90 还会触发七类根因归因。

题集是 25 道真实需求。每道题的"答案"是一份人写的百分制评分表，判定标准是自然语言——比如"某个请求从同步直接返回改为异步过滤后返回"，满足给 12 分，部分满足给一半。谁来判满不满足？LLM 逐项判。

结果：**代码库里能核实的完整跑数只有两次**，相隔一天，得分 69.9（D）和 51.9（F）。两次都是同一个样子——四层评分里有一层整个被跳过，报告末尾写着"归因摘要生成失败"。二十天后，这两份日志作为"清理冗余"被删掉了。

回头看，问题出在四个地方。前三个是技术性的，第四个不是：

第一，判据不能执行。 判据就是"凭什么说这次做得对"。最初我们拿人写的参考实现当答案，逐行比对文件和函数覆盖率——agent 换一种同样合理的架构实现同一需求，就被判成"重大偏离"。后来改成 checklist 逐项打分，主指标就整个落在了 LLM 的语义判断上。这笔账在后来的方法论审计里被算清了：四层评分里确定性最强的那层（跑冻结单测），25 道题里有 22 道根本就没有，它一缺，确定性判分的占比从 65% 掉到 40%，LLM 判分的影响力升到 60%——主指标实质上交给了一个 Cohen's Kappa 只有 0.10 到 0.21 的裁判，而这个区间在统计学上叫"几乎不存在一致性"。事后看，答案其实早就写在同一个代码库里：那份说明文档解释过为什么不给需求和架构文档打分——"LLM 打分误差大于信号"。这个判断当时是对的，只是没人想到，它对代码同样成立。

第二，判分器自己有 bug。 有一份跑数记录，因为 checklist 的提取格式错了，得分是 0；格式修好之后，同一份日志重新判成了 98 分。同一份产物，前后两个分数差了 98 分——那个阶段的分数，还不到能当信号用的程度。

第三，把流水线维持住，本身就是一份全职工作。 那套系统留下的运维文档，讲的几乎全是它自己的事故：日志管道死锁让任务永远显示"执行中"、后台进程收到 SIGTTIN 卡死 23 分钟、子进程变僵尸、端到端单测卡死、子 agent 加载不到插件。没有一条跟"AI 写代码的能力"有关。这里没有谁不用心的问题——要让一条跨进程、跨容器、还要驱动另一个 agent 干活的流水线稳定跑起来，本身就是很硬的工程；只是它硬在了和评测目标无关的地方。

第四，卷子追不上考生。 107 天里，评测目录搬了三次家，判分方式换了三套（跑单测 → 比对参考实现 → checklist 打分）；而被评的那套开发流程自己也在动——评测还在照着旧流程出题的时候，被评的那个命令已经被合并进另一个命令里了。平台越重，它追赶被评对象的速度就越慢，而 agent 的迭代速度恰恰是这个领域里最快的变量。

有个细节我记得很清楚：把评测代码拆出去独立成仓的那天，讲解这套平台怎么用的页面还在更新，设计文档的状态栏写着"已实施，持续迭代中"。当时没有人觉得这件事该停下来，包括我。

同一段时间里还有另一次实验，成本低得不成比例。 为了回答"知识库该怎么让 agent 检索"，我们做了个 A/B：8 道题，两种检索方案各跑一遍，一共 16 次跑数，一天跑完，产物就是会话记录加一份分析。结论直接落成了一次提交——不再让 agent 先读路由文档再跳转，改成直接 grep 知识库，寻路步数从 12 步降到 2 步。那个改动至今还在生产里跑。

十七万行的平台留下的那两个分数，最后没有变成任何一次改动；16 次跑数的小实验，改掉了一个至今还在用的策略。**差别不在工程量，在判据能不能执行、结论能不能变成一次提交。**

还有一件事值得单独记下来。方案本身很早就成型了：三月底的一份思路整理里，pass@k 与 pass^k、题库分成能力评测和回归评测两套、编辑振荡检测（同一文件同一区域被改超过两次）、越界修改检测（实际改的文件对照架构文档要求的文件）、文件命中率（访问过的文件里有多少是该访问的）——都写到了可以直接开工的程度，四天后就有了第一笔代码。可到六月做方法论审计的时候，pass@k 还挂在"待改进"里，重复跑数一次没跑成。这不是没人想做，是那会儿光维持流水线就占满了手——**平台越重，想往里改一个方案就越难**。想清楚一件事只要一次思考，把它落进一个十七万行的系统，是完全另一回事。前言里删掉海量 spec 文档那件事，和这里是一个道理：值钱的是想清楚，不是承载它的那个载体——文档如此，平台也如此。

吃过这个亏，后面给测试线做评测就立了一条纪律：一行平台代码都不写——只有题集、配置、脚本和跑数记录。下面是那条线的推导过程。

（二）测试线的第一版方案，和一个卡死的数字

评测对象很明确：AI 写的测试管不管用。第一版方案（V1）设计了两条轨道：

- **闸门漏斗**：照 Meta TestGen-LLM 的范式，AI 产出的测试连过几道硬闸门——lint 干净、能跑、连跑三次稳定、覆盖率达标；
- **挖空回填**：从测试资产里故意删掉若干条目，让 agent 做覆盖审计，看它能找回多少，召回率就是分数。

几条纪律从第一天就立下：考卷和答案独立存仓，被评 agent 的运行环境里看不到——**考卷不能和课本放一起**；插件版本、业务代码版本、测试框架版本、模型，全部锁死在一个配置文件里；主指标全部确定性判分——**让代码的运行结果当裁判，不靠 AI 互相打分**。

方案跑起来了，闭环也转起来了：跑数 → 看分 → 定向改 skill → 再跑。直到一个数字把一切卡住：某个模块的审计召回率从 100% 跌到 40%，连改两轮 skill，纹丝不动。

改 skill 改不动，就该怀疑分数本身了。扒开评分器复盘，两个问题：

第一，评分器量错了东西。 评分按文件逐个核对，挖掉的条目必须回到原来那个文件才算找回。可会话记录里写得明明白白：那个条目 agent 找到了——只是写进了同模块的另一个文件。评分器把"放错文件"记成"没找到"。40% 量的不是"能不能发现覆盖遗漏"，是"和标准答案的文件划分是否一致"。两轮修复救不回来，因为病根不在 skill，在评分器。

第二，单次跑数在追噪声。 同一批跑数里，一个模块跌 40 个百分点，另一个涨 20 个，方向互相打架。每个模块只挖 5 个条目，召回率按 20% 一跳，±1 条的随机波动和真实回归根本分不开。此前"单点回归 → 定向改 skill"的迭代，一半是在追噪声。

这一课的学费很贵：**评测系统自己也是软件，也会有 bug；而错误的评测信号比没有更危险**——它会把你的迭代引向根本不存在的问题。这已经是同一个坑的第二次：那套重平台栽在判分器的格式解析上，这里栽在文件粒度上。上一篇里我们对覆盖率说过同样的话，这次轮到评测自己。

（三）带着问题做调研，V1 的死因水落石出

数字不可信了，接下来不是修数字，是回头补课。我们做了一轮业界调研，五十多篇论文和基准过了一遍：SWE-Bench 系列怎么构造任务、防污染，Meta TestGen-LLM 怎么设闸门，TestGenEval 怎么算变异分数，SWT-Bench 怎么做 Fail-to-Pass。然后拿业界的尺子量自己的方案，结论不留情面：

四个核心方法里，有顶会论文直接支撑的三个全站得住；唯一自研、没有业界对标的挖空回填，恰恰是塌掉的那个。

它塌得不冤。调研里翻出一项构造效度研究：有实验里模糊测试工具杀死了 100% 的人工注入 bug，却没发现任何一个真实 bug——人造缺陷普遍比真实缺陷好发现。挖空回填考"你能不能猜出我故意删掉的东西"，本质是合成任务；召回率还能靠全量上报刷满，没有任何惩罚。方法论上两头都站不住。

由此推出第二版（V2）的设计主张，一句话：**判据必须是执行结果，不能是合成清单。** 不再问"你找没找到我藏的东西"，改问"你写的测试实际能发现多少 bug"。

Anthropic 后来那篇讲 agent 评测的文章（*Demystifying evals for AI agents*）把判分器分成三类——基于代码的、基于模型的、人工的，并指出基于代码的优势是客观、可复现、易调试。我们第一天立的那条"确定性判分、不靠 AI 互相打分"，和它是同一个结论。

（四）V2：把"能不能发现 bug"变成执行事实

V2 的主指标落在**变异击杀率**上——这正是本部分开头那个问题的直接答案。

AI 写的测试要闯五道闸门：有产出 → lint 干净 → 能跑 → 连跑三次稳定 → **杀死多少变异体**。前四道沿用 Meta TestGen-LLM 的范式，最后一道才是真正的区分闸。为什么不用覆盖率？TestGenEval 有个实测：GPT-4o 生成的测试覆盖率 35.2%，变异分数只有 18.8%——覆盖率能靠无断言的测试刷出来，杀变异体必须有真断言。

变异体就是故意埋进业务代码的单点 bug：比较符翻转、and/or 互换、删掉 `or 0` 这类兜底、把状态写入整句注释掉。测试跑在变异后的代码上失败，算杀死；照常通过，说明断言没盯住这个点。算子里的后四种直接取自我们的代码评审 checklist——那是这个项目真实 bug 的分类学，照它出题，埋进去的 bug 才长得像真实事故。

而埋什么、埋在哪，才是整套评测真正费工夫的地方——**选什么函数，就是在考什么能力。**

V1 的十个函数是按"长相"选的——行数、if 个数、模块分散，还受限于"哪些函数恰好有现成用例"。三个后果：偏科纯计算函数（LLM 最擅长的题型，成绩虚高）；现成用例全是 AI 生成的，没有人写的标准答案，"对标人写测试"这个语义根本不成立；没考虑代码活跃度，一个函数被重构，挂在它上面的 12 个变异体一次团灭。

V2 的选题改成三层：**场景**定业务代表性（发奖、任务、活动、成就、排行榜、聊天六大类），**原型**定失效模式覆盖（校验闸门、资源结算、时间窗口、状态机、兼容兜底、遍历聚合六种，同样来自评审 checklist），**四轴**定难度配平（分支复杂度、依赖构造成本、副作用、函数可见性）。再加六道硬门槛，每条都写明教训来源：函数体 15~80 行（太短没变异空间，太长单测成本失控）、分支密度不低于 3、依赖可 mock、可变异点不少于 6 个。并发和性能两个维度明确不入题——单测考不了。

（五）出题这条链上，人和机器各站在哪

题集建成的过程，是我做过的最干净的一次人机分工。整条链是这样的：

人给方案 → **六个 sonnet 子 agent 从业务代码取素材**（一个场景一个，先读知识库的文件地图锁定范围，再进代码核验，逐条回答硬门槛，交回 34 个候选）→ **人终审定稿**（按场景、原型、mock 轻重三维配平，34 选 16，排除理由全部留痕）→ **脚本造变异体**（按算子轮转配额，保证各类失效模式机会均等）→ **三级验证**（sonnet 初审剔掉明显等价的，参照测试执行验证定生死，人工只复核两边不一致的）→ **deepseek-v4-pro 答题，同一道题答三遍取平均**。

这条链上有三处最见分工：

第一，检索为什么不用脚本。 因为叫 `check_` 的可能是纯转发，叫 `update_` 的反而是真校验——规则匹配不出"这个函数考不考验能力"这种判断，所以这一步交给 agent。但紧接着人要复核它取回来的事实：agent 报"这个文件近一年改了 48 次"，落到候选函数上其实一年不超过 1 次——它报的是文件级，不是函数级。**这就是前言说的那件事的又一次：AI 交付功能没问题，它交付的"事实"本身却常常失真，所以终审前必须补一道函数级核验。**

第二，"什么算真 bug"只有人能定。 造变异体时最麻烦的敌人是等价变异——改了代码但行为不变的"伪 bug"，混进分母就永远杀不死，冤枉所有考生。真正难的不是识别它，是定义它。我们最后定下两条政策：**日志差异不算可观测**（否则会逼出"断言日志输出"的测试，那

不是我们要考的能力）；**可杀性以自然构造为准**（入参和缺省字段可以构造，但要把环境常量 mock 成非法类型、或者传语义荒谬的值才杀得死的，判等价）。这两条是纯粹的价值判断，没有任何脚本能替你拍板，机器只能在拍板之后照着执行。

第三，验证者自己也会错，所以分工不可倒置。 LLM 初审在三个地方栽过：and/or 翻转后短路保护失效、运行时报错其实是可观测的，它判错三个；多分支条件只分析了一支；最有意思的是同一行有两个相同符号时，它的判断理由指着隔壁那个——结论碰巧同向，也算坏留痕，所以审查提示词里硬性要求按列号指认变异点。参照测试自己也有 bug：一次全量验证里 8 个变异体没被杀死，5 个是参照测试写错了（`pcall` 闭包里声明的局部变量，闭包外读到的恒为 nil，断言值恰好与预期重合时会假 PASS，整类变异全漏杀），修好后全部杀死；只有 3 个是真的等价，而且这三个的形态在语法层面完全看不出来。

三级验证之所以不能颠倒顺序，原因就在这里：**LLM 审查省的是机时，执行验证定的是生死。** 最终 184 个变异体终裁，172 个确认可杀入题、12 个等价剔除，每一条判定都带 LLM 初评和人工复核两层留痕，终裁以人工那层为准。

开考前还有一步：用 deepseek-v4-pro 把 16 道题各跑一遍，校准区分度。击杀率跨度 12% 到 100%，宏平均 61%——题不太难也不太简单，这才敢开考。

（六）跑数纪律，和对评测自己的评测

正式跑数 16 道题、每题跑三遍，48 次跑数、约 19 小时机时。统计纪律照业界执行：只看均值不看单次；参照 Anthropic 实测的约 6 个百分点基础设施噪声，差距小于 3 个百分点不下方向性结论，3 到 6 个百分点标"可能显著"，超过 6 个百分点才拿来当改 skill 的依据。这条纪律是被 V1 的教训逼出来的，所以我们把它写进了 runner：默认三遍，只跑一遍的模式仅限空跑。

有一个细节能说明这套跑数为什么可信：统计 agent 有没有自己跑过测试，我们只认会话记录里的工具调用，不认它在文字里的自述。同一条纪律连自己家也没放过——我们那份命令配置声明里写着已经剔除 Task 工具，实际的工具列表里它还在。**声明与事实不符，在自己家里也会发生。**

跑完之后，我们还给评测自己做了一轮评审。最尖锐的一条是：被评的 agent 始终带着完整的测试 skill，缺一个不给 skill 的裸模型条件，"这套插件到底有没有用"答不上来。这条批评对不对？看评测的目的。目的是向别人证明插件的价值，缺基线就是硬伤；目的是优化——找 agent 的短板、指导下一步改进——就没必要答这个问题，多跑一个基线对优化没有任何输入。**所以设计评测前先想清楚目的：目的决定要回答什么问题，也决定哪些问题可以不答。**

（七）五类短板，和一个被数据推翻的分类学

结果里真正值钱的不是分数本身，是失败归因：把没被杀死的存活变异体聚类，聚出五类通病——不测缺省兜底（最大一簇）、不测边界相等值、只断言返回值不查状态写入、不踩非法入参、不枚举模式常量。每一类都能直接转成测试 skill 的通用规则：每个 `x or default` 写一条字段缺失用例，每个不等式写一条恰好相等用例，有状态写入的函数二次调用读回断言。

说句老实话：这些方向，我在推导题集时就已经想到了——选失效原型、设计变异算子，本质就是在回答"AI 写测试最容易漏什么"，答案在出题时就冒出来了。数据的作用是把判断变成带数字的清单、给短板排轻重；方向本身，不是数据告诉我的。

但有一件事是我想不到、只有数据能告诉我的：**我精心设计的那套原型分类，预测不了成绩。** 同挂"校验闸门"标签的两道题，一道 91%，一道 14%；"时间窗口"里是 91% 对 12%，跨度全部拉满。真正决定分数高低的不是原型标签，是"杀死它需不需要构造缺省值、边界值、状态型输入"。以后给题目标难度、做配平，得按这个来。

一套自认为合理的分类学被自己的数据推翻，我认为这是评测最该产出的东西——它比任何一个分数都值钱。

还有一条纪律必须记下来：**这 16 道题既是校准题也是正式题库，照着存活变异体去改 skill，再用同一批题跑分，提升幅度一定被高估。** 要么把改动限制在不含具体函数名的通用规则并在报告里披露，要么留几道题不反哺，当留出集。这叫用考题练考题，是评测最容易自欺的地方，而机器不会主动提醒你。

（八）推到最后，两件事确定了

评测的核心工作是构建题集，不是搭评测平台，也不是搭评测流程。 整个推导走下来，方案的每次进化都发生在题上：挖空清单被推翻、选题从"看长相"改成三层考纲、变异体三级验证、开考前校准区分度。而跑数、判分、报告这些"流程"，自始至终就是一堆脚本——它们的确定性正是它们的全部价值，而确定性不需要架构。凡是能脚本化的环节，都不值得当成工程去建设。这一部分开头那套平台，十七万行几乎全花在了这类环节上。

这个结论有个更大的呼应。姚顺雨老师在《The Second Half》里的判断是：AI 的下半场，重心从"解决问题"转向"定义问题"，评估比训练更重要。**出题就是定义问题。** 我们从工程实践里推出来的"题集是本体"，和他从研究视角给出的判断，说的是同一件事。

还有一条，说出来可能得罪人：**评测确实能帮你改进 agent，但不做评测你一样能优化 agent，而评测的成本，业务团队很难扛。** 帮助是真的——五类短板每类都能转成 skill 规则，原型分类学被推翻这种发现更是白捡的；可评测最贵的那部分产出，和认真分析问题本身的产出高度重合。账是实打实的：一轮校准跑数 500 多块、10 多个小时，正式跑数 48 次约 19 小时机时；题集构建的成本更高，光 token 就花了数千美元，AI 辅助之下人力成本照样下不来。

这里要把两件事分开：**"是不是常"说的是资产保值，"值不值得做"说的是投入产出。** 题集确实是常——真实 bug 变成的变异体、按真实事故分类学定的考纲、三级验证的流水线、那两条政策口径，换什么模型都接着用。但"保值"不等于"现在就该投"。什么时候值得上评测？要在两个方案之间做高成本取舍，或者要向别人证明改进真实存在。日常迭代，靠真实需求反复跑、靠会话分析定位短板（第一部分就是这么干的），一样走得动。

第四部分：人退到决策点

> **本部分要点**

> - AI 接过写代码之后，人退到决策点上，不再站在产出线上；验收是决策点最密集、也最麻烦的一处。
> - 问题不是“要不要逐行 review”，是哪些必须人看、哪些交给机器；更麻烦的是有些错误人肉审查在原理上就发现不了——有些事实只有机器取得到。
> - 能信 AI 写的测试吗？恰恰要重点检查测试：离线靠变异击杀（第三部分），在线靠平台独立重跑（第五部分）。
> - harness 之于 coding agent，犹如 IDE 之于人类程序员；会话内的约束 AI 有办法绕，会话外的流水线它绕不了。

机器那一端顶住了、也量得出了，现在可以回答标题里那个问题了：AI 把写代码这件事接过去之后，人在流程里还干什么？

我的答案是：**人退到决策点上，不再站在产出线上。** 需求和验收是决策点最密集的两处。

需求那一侧，前言已经交代过了：文档可以全部交给 AI 落笔，想清楚这一步不行。AI 能把方案写得很完整，但"这条路走不走"它替你答不了——它会把你说的每个方向都论证得头头是道。所以这一节专讲另一侧，验收。它更麻烦，因为验收要面对的不是一份方案，是几万行代码。

（一）为什么不是"人应该看得更快"

人验收 AI 的产物，无非两条路：评审代码，测试验证。两条路各有一个尖锐的问题。

问题一：还有必要逐行 review 代码吗？

这个问题今年吵得很凶，而且给出激进答案的，恰恰是两位最有资格保守的人。

Redis 之父 antirez 那篇文章的标题就是《控制想法，不控制代码》。他的判断是：**如果你掌控了软件的想法，逐行看代码是次优的、往往无意义的。** 三条理由：一天 5000 行，谁看得完；LLM 很擅长写局部最优的代码，不擅长大的设计取舍，所以逐个函数扫过去收益不大，更该做的是把你心里的设计讲出来，再反过来问它"这部分的设计到底是什么、怎么运转的"，然后判断这个模型对不对；工作日只有 8 小时，读代码就是在挤占今天最该做的事——想清楚这软件要往哪走，以及大量 QA。他自己确实还在逐行审 Redis 的 AI 代码，但明说这"多半没有意义"，坚持做是出于对用户的尊重：Redis 是公共基础设施，很多人会打开文件手改。

他还有一句很扎的话：那个内存优化 PR，前沿模型的 review 会比他自己的 review 发现多得多的错误和微妙竞态。 这句话不是猜想，我们的评审服务每天都在验证它——两分钟轮询一次提交，多趟 AI 评审，结论直接推给提交人。

《代码整洁之道》作者 Robert C. Martin 说得更直接："我目前的策略是完全不读 agent 写的任何代码。只有这样，我才能真正利用它们带来的生产力提升。" 但下半句才是重点——他在 agent 周围架了一圈极严的约束：单元测试、Gherkin 验收测试、QA 流程、质量指标、**变异测试**、覆盖率，代码必须跑完这一整套闸门，他才对它有信心。请注意清单里的变异测试：这篇文章第三部分推导了一个季度才落定的主指标，就是它。

而他后来补的一条推文划出了更精细的边界：agent 写的单元测试他不看；agent 写的验收测试和 QA 流程他亲自看，关键功能全审、普通功能抽查，此外定期做一次最终的人工测试。换成这篇文章的说法：**他放掉的是实现，握住的是判据。**

反方的质疑同样有力，而且正好戳在这篇文章花两个部分去解决的地方：AI 遵循指令并不稳定，凭什么相信它一直待在护栏里面？**如果它擅自改测试来迁就自己的实现，或者制造出"测试通过"的假象，人怎么察觉？** 还有更根本的一条——测试只能证明程序满足了被写进测试的那些条件，它证明不了这些条件本身完整、正确。

所以我的看法是：问题已经不是"要不要 review"，而是**哪些必须人看、看到什么程度、哪些可以交给机器**。这两位的答案，方向和我们一致——关口从人的眼睛挪到机器的闸门。

但我还想补一条他们都没细说的，它比"量太大"更麻烦：**有些错误，人肉审查在原理上就发现不了。**

举一个我们真实撞上的例子——就是第二部分黑盒事故清单里的第二件，这里完整讲一遍。我们用 CLI 批量生成中文技术文档，41 条任务、37 份产出，其中 **36 份损坏，累计 370 处**：汉字变成 U+FFFD，而且是一个字的三个字节各自变成一个 U+FFFD。根因在 CLI 拼接工具调用参数时把 UTF-8 多字节字符按字节切断了，不是模型写错字。

这个 bug 对使用者极其隐蔽：**文件是合法 UTF-8，正文读着通顺，只是偶尔缺一个字。** 不做字节级排查，你只会觉得"模型偶尔手抖"。让人一份份看，看一百遍也看不出来。

同类的还有：测试声称跑通而覆盖率为空；断言被悄悄放松之后测试依然全绿。这些都不是"人不够努力"，是人的感官在这些维度上取不到事实。

所以"机器提供事实"不是为了省人力，是因为**有些事实只有机器能取到**。

那次事故最后变成了一道门禁：产物入库前扫 U+FFFD，命中直接拒绝，需求文档、知识候选、评审报告一视同仁。人看不见的那类损坏，从此不必再靠人看。

问题二：能信 AI 写的测试吗？

测试验证是最硬的验收——上面那套"不读代码"的策略，整个压在它身上。但这里有个死结：测试代码也是 AI 写的，而 AI 为了跑通测试会悄悄放水——断言改松一点，边界少测一条，测试就绿了。Uncle Bob 说单元测试他不看，前提是他有变异测试和一整套指标在后面兜着；**少了那层兜底，"不看单测"就只是把信任凭空交出去。**

所以要重点检查的，恰恰是测试。怎么检查？答案分两半，两半都要落成可执行的事实：

- **离线的一半**：不问"你写的测试覆盖了多少"，改问"你写的测试实际能发现多少 bug"——往业务代码里埋真实形态的缺陷，看它杀掉几个。这是第三部分。
- **在线的一半**：不信 AI 自己跑出来的那份报告，入库前用平台的容器独立重跑一遍，把逐条执行结果摆到审批页上。这是第五部分。

（二）harness 之于 coding agent，犹如 IDE 之于人类程序员

一直有种说法：模型越强，harness 该越薄。前言里说了我为什么认为这是想当然。还有一种对称的说法：harness 越厚，agent 越笨。两句话把三种不同的东西揉成了一个词。第一部分已经拆开讲过，这里再补一个类比。

把 AI 当成人就想明白了。用记事本写代码的人确实存在，也确实厉害，但实际开发大家都用 IDE，还有很多人乐于打造独属于自己的 IDE，因为那是最高效的方式。**程序员越强，对趁手工具的要求越高，不是越低。**

回顾人类怎么一步步提高编程效率的，就两个方向：

1. **改进语言**：OOP 给了拆解复杂度的思维范式，类型和 GC 减轻了心智负担。
2. **完善工具链**：LSP、lint、格式化工具、静态检查、性能分析，每一样都在降错误率。

类比到 AI，答案是一样的：**为 AI 量身定制开发工具链。** AI 友好的工具，最好简单到自动触发、即时反馈，像第一部分那个 trace 工具。

如果你守着一个历史悠久的大项目，历史债堆成山，技术栈又换不掉，挑战会大很多——但方向不变。反过来也别以为通用技术栈就能省掉这一层：前言里那个平台就是通用栈，一样得建这些东西。

（三）会话之外必须有一层

工具之外还有一层：**会话之外的 check 流水线**——规则式的质量门禁，语义式的 checklist。

为什么必须有会话外这一层？因为 AI 的训练是应试式的，为达目标不择手段，它目前还理解不了价值本身。它干得出荒唐事：gpt5.6 被曝跳出沙盒直接抄测试答案；为了让测试通过，直接改测试本身。**会话内的约束它有办法绕，会话外的流水线它绕不了。** 这种兜底 check 比人类注意力便宜得多，多多益善。

所以人不是要看得更快，是要退到只做判断——把核对交给机器，把绕不过的规则交给流水线。

第五部分：把判据变成流程

> **本部分要点**
>
> - AI 的单次产出不可靠，人的单次注意也不可靠——用多重独立的嵌套环叠加出可靠性。
> - 环的接口只许传事实：生产者不是写者，失败必须有名字，事实源唯一且可推导。
> - 不信上一环的自报：平台用自己的容器独立重跑，把事实摆到审批页上，判断留给人。
> - 资产不是文件，是活的生命周期；人不是环外的验收员，是环里的节点，平台的职责是降低人做判断的成本。
> - 该不该自建：可替代的层本来就没自建；自建的是判据、资产模型、环的接线——凡是本体都不通用。

前四部分讲了怎么让 AI 顶得住、怎么量它干得好不好、人退到哪里。最后一部分讲：怎么把"机器提供事实，人做判断"从一句原则，变成每天都在跑的研发体系。

这就是前言那个平台的由来。先声明一句：平台还在建设中，这一节讲的不是建成报告，是建设思路。一个这么重的平台该不该由业务团队自建、会不会被中台轻易替代，是它最容易被质疑的地方——这个问题先挂着，答案放到最后一节。思路的结论可以先给：**这个平台承载的不是功能，是环——人、agent、资产编织成的、转速各异的嵌套环。** 整个推导从两个不可靠开始。

第一个不可靠：AI 的单次产出。 前言已经说过，出错的绝对数量不由单次准确率决定，由累积量决定。一次会话里写得再好，也不能指望它无瑕。

第二个不可靠：人的单次注意。 这条更不容易承认，但同样是事实。代码不是人写的，大量细节超出人的观测；一次长会话里，人的注意力会疲劳、会变形，判断质量跟着产出量一起往下走。让人在一个环里盯到底，和让 AI 在一个环里写到底，是同一种侥幸心理。

（一）环为什么要嵌套

两个不可靠合在一起，推出的不是"更认真地检查一次"，而是**用多重独立检查叠加出可靠性**。

我们的做法是把环分成大小两层。一次会话里的"开发加自测"是小环——AI 写，AI 自己跑，人在会话里看。会话结束，大环启动：agent 自动做代码评审，过各类 checklist 和静态检查，跑更全面的测试验证。不能指望在一个小环里让 AI 写出完美无瑕的代码，也不能指望会话里的人盯得住。

大环要成立，有两个设计点。

第一，每个环必须独立。 评审的触发是轮询提交，与会话无关；测试的验证用平台自己的容器重跑，与生产者的自报无关。触发源和被验对象之间没有因果关系，这个环说出来的才是新话，不是把上一环的结论复述一遍。

第二，大环不消灭注意力问题，它拆解注意力问题。 一次长会话要求的是持续的、发散的注意，那恰好人最不擅长；多个环把同一件事拆到多个场景、多个时机触发，每个场景主题单一，需要注意的点集中，注意力效率完全不同。而且同一个问题会被不同角度的环反复提醒——被反复暴露的问题，比被凝视一次的问题更容易被发现和修复。

环里还有一类不占注意力的角色：**闸门**——确定性的规则和 checklist。它们不消耗人的注意力，只把例外递到人面前，所以多多益善。唯一要守住的是第一部分那条标准：闸门的判据必须是事实。判据是事实，闸门做硬拦截；判据是代理指标，最多做提醒。闸门吃掉常规，人只看例外，环才转得动。

（二）环的接口：只许传事实

环多了，问题就转移到接口上：环和环之间传什么？我们的答案是一条铁律——**环的接口只许传事实，不许传声明。** 接口处传一次声明，下游所有环都在验一个假东西。

这条铁律落成三个机制。它们都是从测试用例这条环上长出来的——那是第一条转完整圈的环——就以它为例。

机制一：生产者不是写者。 生成侧只提交候选，写仓库的权利全部收归平台：平台是唯一写者，每个受治理的仓一条串行队列——AI 负责快，人负责对。

链路统一成：AI 生成候选 → 人工审查 → 串行入库 → 沉淀为资产。审批不只是通过或拒绝：人可以改内容、挪归属、整条丢弃，入库时平台会把不再被引用的公共代码一并裁掉，不让垃圾进仓。审批是要花时间的，这期间仓库可能已经被别的入库推进了，所以入库不是追加，是合并——能自动合的自动合，合不了的标出来交给人。**人只在真冲突时介入，其余交给机器。**

机制二：失败必须有名字。 环是跨天、跨容器、跨进程转的，卡住的时候，人要能一眼看到环卡在哪一节。

这条是被一次事故按着头教的。一批用例的入库长期卡在"正在合并"，前后四次恢复都失败。查下去发现三层原因叠在一起：底层仓库对象有"有没有实际改动"这个能力，但生产实现没有把它透传上来，内核探测失败后默认"有改动"，于是去提交一个空 commit；普通错误没有对应的失败状态，网关只记日志、不改状态；重启后的恢复逻辑只会重投"正在合并"的任务，同一个空 commit 循环自己养活自己。

最讽刺的是测试：**替身实现直接把那个可选能力实现了，所以测试永远绿，测不到生产漏接线。**

修法有三处：把那个能力变成必需方法，漏接就编译不过；没有实际差异就跳过提交直接收口；普通失败、超时、panic 一律进一个可重试的"合并失败"态，停机取消保留原状态，恢复逻辑不再自动重试失败态。状态机由此变成七态：待审、正在合并、已入库、已丢弃、冲突、推送失败、合并失败。**最后一态是事故买来的。** 一个没有名字的失败，对人来说等于没有事实；对一个环来说，等于这个环消失了——它还在转，只是永远转不到头。

机制三：事实源唯一，而且要能被推导。 环与环之间不能传两份事实，也不能传"它说是这样"。

AI 生成的测试和覆盖率事实怎么关联？我们的选择是**按路径规范推导**：用例文件路径 `ucases/<被测文件路径>/<被测函数名>_unit.lua` 就是被测函数归属的事实源。为什么不让 AI 在元数据里声明？因为**声明会撒谎，路径不会**。这句话要补一个诚实的注脚：现网有两千多条单测的归属字段本来就是空的，我们其实没得选；而代码里至今保留了一条"库里已有值就先用它"的分支，也就是说 AI 真填了，还是会被信。这条路只走了一半，另一半是明确的技术债。

覆盖率事实还有两条判断，都是被自己打脸打出来的：

- **多用例合并按行取并集**
  单测逐条单独跑，每条快照只反映自己覆盖的分支，取任何一条都会低估。
- **"没跑过"和"跑了没覆盖"严格区分**
  分母只含实际跑到的函数，两种情况都不显示 0%。

第二条就立在平台曾经把"采集失败"显示成"没有数据"的废墟上。**上一篇里我们说错误的覆盖率信号比没有更危险，第三部分那两次是评测系统自己有 bug，这里是同一个教训的第四次——而这次撒谎的是"事实"本身。**

（三）环的入口：不信上一环的自报

接口管"传什么"，入口管"信不信"。生成侧的 AI 自己跑过测试，也上传了覆盖率报告。这份结果不能信——它可能声称成功而覆盖率为空。

所以平台用自己的容器池，对待审批的用例独立重跑一轮，逐条执行结果直接摆到审批页上，当审批材料。**结果仅供参考、不阻塞审批——机器提供事实，人做判断。**

执行容器池的设计也顺着环的思路：池成员不是平台建的，是管理员把自己的开发容器贡献进池子。每个容器若干执行槽位，槽位内严格串行、槽位间并行；执行记录只追加不修改；失败换槽位重试。手动触发的跑在触发者的私有池，平台自动验证跑公共池，互不挤占。

（四）环上的货币：资产要被治理，而且是活的

环和环之间真正在流动的东西，是资产。需求、用例、知识——它们进仓前的形态一律是"候选"，审批是货币的发行纪律：人做过判断才入库，入库才成为资产。不治理的情形我们见过：AI 生成的用例散落各处，没人知道哪些能跑、哪些已经过时；评审意见发完企微就沉了，同样的误报下周再来一遍；知识沉淀不设关卡，错误经验写进正式知识，比没有知识更危险。

人的判断本身也是数据。用例为什么被丢弃、评审为什么被标无价值、知识候选为什么被拒，这些原因和审批结果一起结构化落库——它们是回流给生产端的信号：哪类候选总被丢弃，上游的提示词和 skill 就该改哪里。但回灌要克制：**先把人的判断攒成可信的数据，再谈自动回灌。** 拿一批没经过校准的标签去改提示词，等于用脏数据训练上游——那就是第三部分那个错误评分器的翻版，只是这次污染的是生产者。

资产不是文件，是活的生命周期——这是"为什么必须是平台而不是仓库"最硬的回答。结合版本周期，一个需求的上下文至少要支撑两个版本：一个上线版本，一个在研版本。需求上线后出 bug 要 patch，而 patch 期的修改模式和开发期完全不同：特别在意最小改动、兼容性、数据修复和可观测。这几条恰好全是前言说的那种"价值观型"约束——AI 交付功能的能力强于交付纪律的能力，所以它们不能写进提示词指望自觉，要做成挂在需求状态机上的判据。仓库存得下文件，存不下一个要活过两个版本、两期纪律完全不同的上下文。**资产的活法，就是环的转法。**

（五）人是环里的节点，不是环外的验收员

平台奠基时立过一条边界：只做外环，不做云端编码环境，警惕膨胀成 Codespaces。后来我们自己推翻了它，做了 AgentUI——浏览器里的会话工作台，直连容器内的 agent CLI，权限审批、提问、干预都在页面上完成。

推翻的理由比原来的边界更本质：**人不是环外面的验收员，人是环里面的节点。** 节点要干活——批权限、定稿、审批、随时和任意 agent 对话——平台就得把交互做进来。这也是第二部分那个需求在平台尺度上的重演：workflow-engine 给不了的掌控感，环系统必须给。

人在环里，紧跟着的问题就是：人怎么跟得上环的转速。这件事的本质不是让人读得更多，是**降低人做判断的成本**——用一切手段，让人能高效、轻松地完成判断。手段没有尽头，目前做的是三个例子。

第一，产物形态要从 Markdown 走向 HTML。 一百行以上的 Markdown 我自己都读不完，更别说让别人读。HTML 能承载表格、SVG 图示、折叠、导航，还能一键分享。Anthropic 的 Claude Code 团队专门写过一篇讲这件事，他们给出的理由和我一样：用 HTML 之后，人重新回到了循环里（feel much more in the loop）。而且他们强调了一个我认为最关键的做法——自定义的 HTML 界面必须以"导出"结尾：copy as JSON、copy as prompt，你在界面上做的判断能一键变回给 AI 的输入。回路因此更紧。我们目前做到的只有一部分：测试报告、覆盖率报告和 trace 语义视图已经是自包含 HTML，人不必读 JSON；需求文档、知识条目、评审意见还都是 Markdown。

第二，万物可标注，标注回灌 AI。 这条平台里还没有，但在 `/dev` 流程的插件里已经跑通了一个小回路：人在方案上划词批注，批注当场变成结构化输入回给模型。方向是清楚的——人的判断是这套系统唯一的外部输入，机制的职责是让它不流失、可复用。

第三，trace 报告要可视化，而且数字不许模型碰。 trace 语义视图的设计原则值得单独说：LLM 只负责命名、归组、写文字解释；所有数字由确定性脚本从原始证据重算；守恒校验不过，HTML 就退回原始全量视图。把"机器提供事实、模型只做表达"用在可视化上——可视化不是把 JSON 涂漂亮，是在不牺牲真实性的前提下让人一眼吸收。

（六）环要铺满研发周期

到目前为止讲的环，大多围着测试和评审转。但环的目标从来不是某一条链路，是整个研发周期：需求、开发、评审、测试、上线、patch，每个环节都值得有环。环的骨架只有一副——触发、生产、闸门、人审、入库、复用——往哪个环节铺，都是这一副骨架的实例。离我们最近的三个，挂在研发周期最前段：

- **需求预分析**：需求单分配后 agent 立刻开工——大致方案、策划案没写明的细节、影响面与工作量评估。这条环最克制的地方是产出形态：

待澄清清单是核心价值，方案只是 AI 的一种自洽——不标注清楚，看的人会把它当推荐。

- **bug 预分析**：是不是 bug，客户端、DS 还是后台的问题，原因是什么，怎么修。它有一个别的环没有的奢侈品：这个 bug 最终真的会被修，真值免费回流，判对率可以统计、可以校准。
- **评审预分析**：人工 review 之前，agent 先自动过一遍，排查常见形态的 bug，把人的注意力留给真正需要判断的地方。

（七）业务团队该不该自建这么重的平台

环铺得越满，那个绕不开的问题就越大：**这么重的平台，业务团队有没有必要自建？会不会被中台轻易替代，投入打水漂？** 我们的答案分两层。

第一层：可替代的层，本来就没自建。 容器用 AnyDev，CLI 用现成的，需求单从 TAPD 同步。真正自建的是三层中台给不了的东西：**判据**（变异算子来自我们的评审 checklist，patch 纪律来自我们的版本制度）、**资产模型**（路径规范、知识模块、需求文档体系）、**环的接线**（谁在什么时机收到哪个例外）。中台能给壳——算力、会话、通用 CI；给不了本体。这和第三部分是同一个结论的一体两面：题集是本体，判据是本体，资产模型是本体——**凡是本体都不通用，凡是通用的都不是本体。** 哪天通用平台真把壳做得更好，换掉就是；自建的部分恰好是按"不可替代"筛过的，这不是巧合，是建设原则。

第二层：投入沉在哪，决定打不打水漂。 平台最值钱的产出不是代码，是判据、题集、资产模型、环的设计，和一路攒下的失败史——它们跟着团队走，不跟着平台代码走。前言里删掉的 spec 文档和这里是同一个道理：载体可弃，想清楚的东西省不掉。就算明天整个平台被替代，这些本体一件都不会浪费。

这个平台是研发体系的运行时：人、agent、资产编织成转速各异的嵌套环，人从产出线退到环的节点上只做判断，机器在环的接口处提供可验证的事实，资产在环间流动并增值。

结语

五个部分讲的是五件事，但回头看，每一件里的动作都一样：把某个判断从"听它说"改成"去验它"。不信状态符号，改看超时有没有消息；不信活体成员还在，改认磁盘上的产物；不信 AI 自己跑出来的那份报告，改让平台用自己的容器重跑一遍；不信元数据里的归属声明，改从文件路径推导；不信我自己设计的那套失效原型分类，改看变异体到底被杀死了多少。

一句话：**不信声明，信事实；不信自觉，信机制。** 它和开头那句"机器提供事实，人做判断"是一体两面——后者说的是分工，前者说的是这个分工凭什么立得住：机器那一半交出来的必须是可验证的事实，不是听起来可信的说法。上一篇说"硬门禁比软约束可信"，这一篇把它贯彻到了底。

还有一条更硬的教训，这一年里它出现了四次：上一篇说错误的覆盖率信号比没有更危险；第三部分那套重平台的判分器因为格式解析出错，把同一份产物判出两个差 98 分的分数；紧接着自研评分器把"放错文件"记成"没找到"；第五部分发现平台把"采集失败"显示成"没有数据"。**同一个判断，我们在四个地方各犯了一次。** 所以"机器提供事实"不是一句口号，它的难点从来不在"人做判断"这一半，而在"机器给出的那个事实，本身是不是真的"。提供事实的机器，自己也要被验证。

最后回到开头那个平台。它是纯 AI 交付的：三个月，七个服务，三十多万行代码，人一行没写；它也会出事。这两件事不矛盾——**AI 的产出能力已经够用，而正因为它产得又多又快，兜底比以前更必要，不是更不必要。** 不管什么技术栈，都是这样。

模型每季度都在变强，提示词、编排、评测数字都会随之作废——它们是流。约束型尤其如此：补偿模型当下缺陷的那些 SOP，模型一换代就该删。但另一些东西，模型再换代依然值钱：把问题想清楚的那个过程（连它产出的 spec 文档都能删，它本身省不掉）、从上百次失败会话里提炼的运维纪律、对 runtime 主权的判断、框架层那些不占模型注意力的能力、验收的杠杆、评测题集、治理链上的每一条判据——它们是常。**做 AI 工程，就是在湍急的流里，找到并守住那些常的东西。**

观千帆而识其流，历万变而守其常，承其重而赴其远。

以上是这一个季度摸出来的东西，不少还在半路上，也还在改。与诸君共勉——有疑问、有不同看法，欢迎评论区随时交流。

#AICoding #AI #代码 #提示词 #大模型 #技术





阅读 1.8万

留言

写留言

还没有留言，来写首评